

Università degli Studi di Napoli Federico II

Scuola Politecnica e delle Scienze di Base

Corso di Laurea in Informatica

UninaMoneySplit

Relazione Tecnica di Progettazione della Base di Dati

Esame: Basi di Dati

Candidati : Paolo Augusto , Nicola Barricelli

1. Introduzione

1.1 Scopo del Documento

Il presente documento descrive l'analisi e la progettazione (concettuale e logica) della base di dati per la piattaforma UninaMoneySplit, sviluppata come progetto d'esame per l'insegnamento di Basi di Dati. La relazione ha l'obiettivo di illustrare le scelte fatte da noi studenti che ci hanno guidato dalla transizione dai requisiti informali espressi in linguaggio naturale fino al modello relazionale definitivo pronto per l'implementazione sul DBMS.

1.2 Interpretazione della Traccia e Analisi dei Requisiti

Dalla traccia abbiamo interpretato il Sistema proposto come uno strumento centralizzato per la semplificazione e trasparenza dei rapporti di debito e credito per spese effettuate da soggetti che si ritrovano all'interno di gruppi come coinquilini, colleghi o gruppi di viaggio.

Dall'analisi della traccia sono emerse tre caratteristiche fondamentali che il database deve considerare:

1. **L'isolamento dei contesti:** Ogni Spesa deve essere confinata all'interno di un perimetro ben definito (il Gruppo) per il quale l'utente ha una partecipazione attiva e il

corrispondente invito, in modo che un utente possa avere un saldo indipendente a seconda del contesto.

2. **La distinzione della natura delle spese:** Il sistema deve saper discriminare da solo quando una spesa è ad esclusivo carico di chi la effettua (quindi una spesa personale, utile per il tracciamento statistico individuale) e quando invece deve essere condivisa fra tutti gli utenti (spesa comune, che richiede una divisione in quote).
3. **La tracciabilità dei flussi di movimento:** Ogni movimento finanziario di uno specifico importo effettuato con l'obiettivo di estinguere un debito (il singolo Movimento), inoltre ogni singolo movimento deve corrispondere ad una e una sola quota cioè un movimento viene effettuato per pagare uno ed un solo debito.

2. Descrizione del Progetto e Assunzioni

2.1 Attori del Sistema

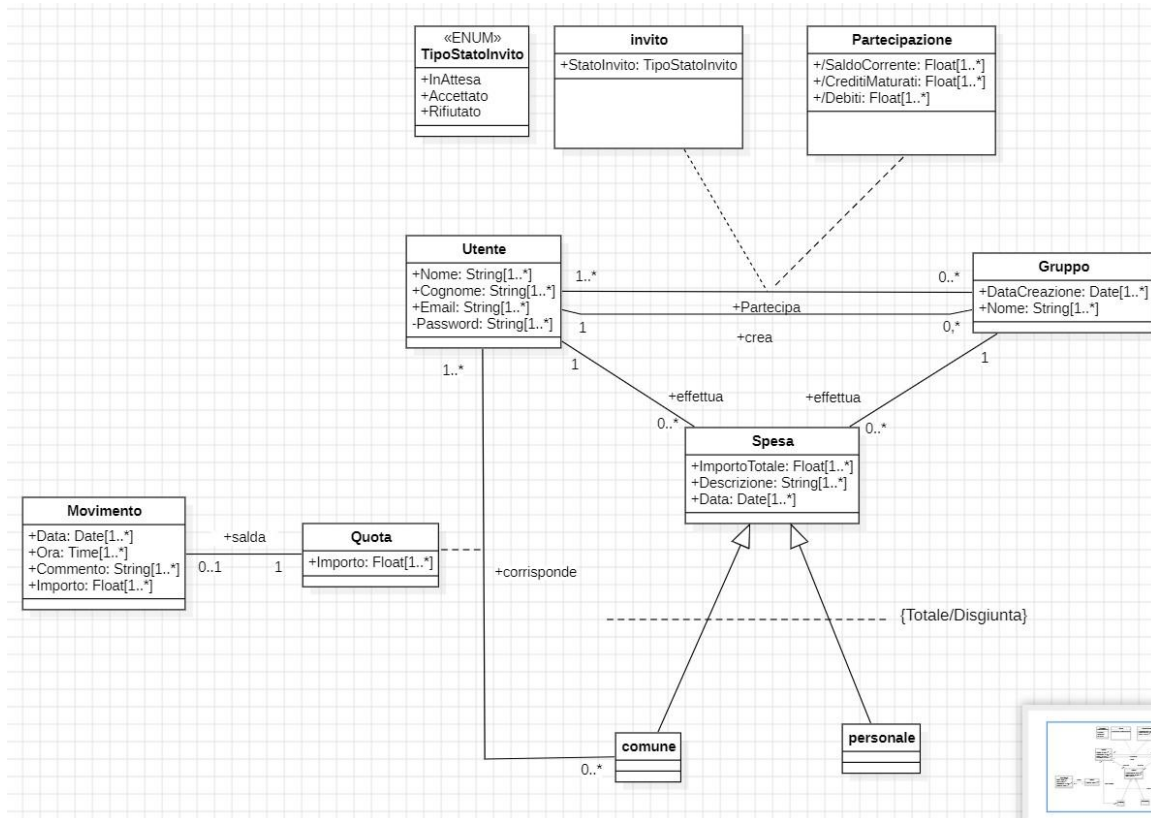
- **Utente Registrato:** L'entità anagrafica di base, identificata univocamente dalla propria e-mail.
- **Proprietario / Creatore del Gruppo:** L'utente che definisce il contesto in cui potranno essere condivise le spese. Basandoci da quanto estraibile dalla traccia, a questo attore è riservata la capacità di generare proposte di invito per altri utenti.
- **Partecipante Attivo:** Un utente la cui associazione con il gruppo è confermata (invito accettato). Solo in questo stato l'attore può essere coinvolto nella condivisione delle spese e di conseguenza la possibilità di dover pagare o avere debiti.
- **Pagatore della Spesa:** Il membro che anticipa la somma di denaro per conto del gruppo o di sé stesso che sarà quindi il creditore della spesa.
- **Debitore della Quota:** Il membro che ha maturato un debito attraverso una spesa comune per un altro membro che ha pagato la suddetta.

2.2 Assunzioni sui Dati e Regole di Business

- Un gruppo deve necessariamente fare riferimento a un solo creatore e se quest'ultimo viene rimosso dal Sistema il gruppo non può esistere (per come è stato realizzato il sistema, non ci permetterà di cancellare gruppi se nel database è ancora presente il creatore a cui fa riferimento).
- L'invito è considerato come una fase preliminare e di transizione; i dati come saldo, debiti e crediti non appartengono all'invito, bensì all'attiva partecipazione del membro nel gruppo.
- Il mittente di un invito deve coincidere con il creatore del gruppo; questa scelta di progettazione ci permette di non appesantire la tabella degli inviti aggiungendo colonne ridondanti, ma invece ricavando l'informazione mediante join referenziale con la tabella Gruppo, si suppone quindi che ogni istanza di invito sia stata generata dal creatore del gruppo.
- Un movimento di rimborso è vincolato ad estinguere esclusivamente una sola quota di debito per volta.

3. Progettazione Concettuale

3.1 Class Diagram



3.2 Analisi delle Proprietà della Gerarchia Spesa

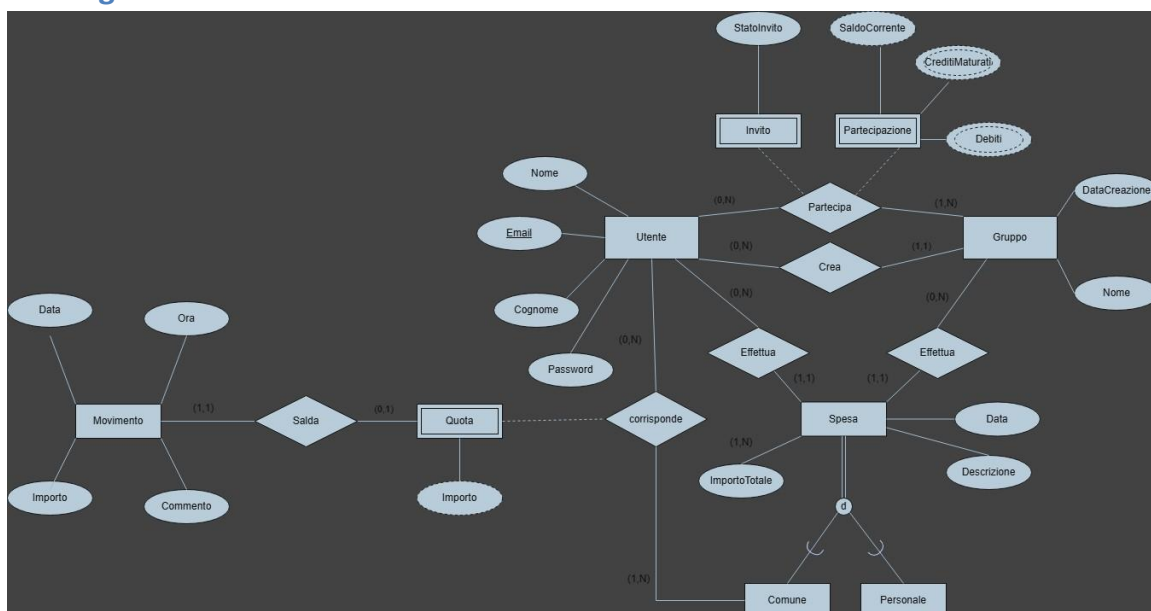
Osserviamo che nel diagramma si trova una gerarchia

SPESA

Spesa si specializza in comune e personale, Riteniamo che questa gerarchia sia **totale e disgiunta**

- **La gerarchia è Totale (t) perché:** non esistono casi in cui una spesa non sia una delle specializzazioni definite, quindi, sarà per forza definita o come comune o come personale
- **La gerarchia è Disgiunta (d) perché:** non è possibile che una spesa possa essere allo stesso tempo sia personale che comune ma può solo esserne una fra le due

3.3 Diagramma Entità-Relazione



3.4 Dizionario delle classi

Dove non esplicitamente indicato si intende che la molteplicità è [1..*].

Classe	Descrizione	Attributi
Utente	Rappresenta una qualsiasi persona che utilizza la piattaforma UninaMoneySplit.	Email: Indirizzo e-mail dell'utente, funge da identificatore univoco naturale. Nome: Nome dell'utente. Cognome: Cognome dell'utente. Password: Stringa codificata per l'accesso.
Gruppo	insieme di utenti creato per gestire le spese condivise (es. coinquilini, viaggi).	Nome: Nome assegnato al gruppo. DataCreazione: Data di creazione del gruppo.
Spesa	Classe padre che rappresenta una spesa effettuata da un utente.	ImportoTotale: costo totale della spesa. Descrizione: descrizione o motivazione della spesa. Data: Data in cui la spesa è stata effettuata.
Personale	Specializzazione di Spesa. Rappresenta una spesa effettuata esclusivamente dall'utente che ha pagato.	

Comune	Specializzazione di Spesa. Rappresenta una spesa da suddividere in quote tra i membri del gruppo.	
Movimento	Transazione finanziaria eseguita per rimborsare una quota di debito.	Data: Data di esecuzione del rimborso. Ora: Orario della transazione. Commento[0..1]: Eventuale nota allegata al pagamento. Importo: Valore monetario trasferito.
Partecipazione	Classe di associazione che descrive i dettagli dell'appartenenza di un utente a un gruppo.	/SaldoCorrente: Attributo derivato, rappresenta il bilancio netto dell'utente nel gruppo. /Debiti: Attributo derivato, lista totale delle quote da pagare. /Crediti: Attributo derivato, lista delle quote da ricevere.
Invito	Classe di associazione che traccia lo stato della proposta di ingresso in un gruppo.	StatoInvito: Valore enumerato che indica lo stato della proposta {InAttesa, Accettato, Rifiutato}.
Quota	Classe di associazione che rappresenta la singola frazione di debito derivante da una spesa comune.	Importo: Valore monetario della frazione di debito che spetta all'utente.

3.5 Dizionario delle associazioni

Vogliamo far notare che anche se non è presente nel nostro diagramma delle classi la relazione invito, abbiamo preferito tenerla nel dizionario delle associazioni, per enfatizzare i ruoli che copre la relazione partecipa che non controlla solo la partecipazione ma anche la fase preliminare che sarebbe l'invito

Nome	Classe1	Classe2	Descrizione
------	---------	---------	-------------

Crea	Utente	Gruppo	Identifica il creatore del gruppo. Ogni gruppo ha 1 creatore; un utente può creare N gruppi. <i>crea creato_da</i> <i>1 0..*</i>
Partecipa (invita)	Utente	Gruppo	Traccia la proposta di inserimento di un utente in un gruppo. <i>riceve inviato_a</i> <i>0..* 0..*</i>
Partecipa	Utente	Gruppo	Gestisce l'appartenenza attiva di un membro al gruppo. <i>membro_di include</i> <i>0..* 1..*</i>
Effettua U/S	Utente	Spesa	Collega la spesa all'utente che l'ha materialmente pagata. <i>paga pagata_da</i> <i>1 0..*</i>
Effettua S/G	Spesa	Gruppo	Indica a quale gruppo corrisponde la spesa che è stata registrata. <i>registrata_in contiene</i> <i>1 0..*</i>
Corrisponde	Utente	Comune	Associa agli specifici membri del gruppo le loro rispettive frazioni di debito da pagare, derivanti da una spesa comune. <i>deve_pagare divisa_tra</i> <i>0..* 1..*</i>

Salda	Movimento	Quota	Collega lo specific movimento di rimborso alla specifica quota debitoria che va ad essere pagata. <i>paga_quota</i> <i>saldato_da</i> <i>1 0..1</i>
-------	-----------	-------	--

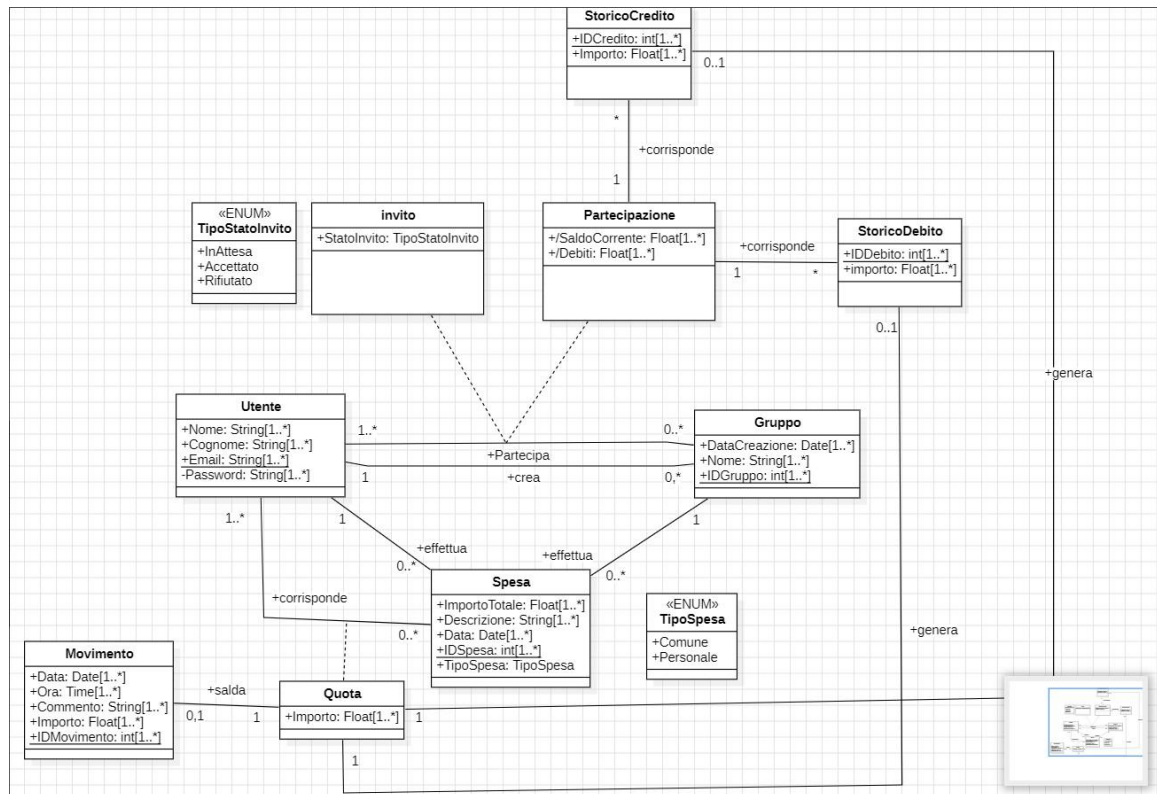
4. Ristrutturazione dello Schema Concettuale

In questa sezione ci focalizziamo sulla ristrutturazione dello schema concettuale per far sì che nella prossima sezione saremo in grado di procedere facilmente con il mapping dallo schema concettuale ristrutturato a quello logico

4.1 Giustificazione delle Scelte di Ristrutturazione

1. **Appiattimento della Gerarchia Spesa:** Le sottoclassi Comune e Personale non avevano attributi propri. Perciò abbiamo optato per la strategia di accorpamento nel padre: abbiamo eliminato le sotto classi rimanendo solo la classe spesa, alla quale abbiamo aggiunto l'attributo TipoSpesa (Enum: {PERSONALE, COMUNE}) utilizzato per discriminare lo specifico tipo di spesa.
2. **Scelta degli Identificatori (Chiavi Surrogate):** Abbiamo ritenuto opportune aggiungere delle chiavi surrogate di tipo intero come IDGruppo, IDSpesa e IDMovimento dato che fra gli attributi che già possedevano non c'è ne fossero di cui avremmo potuto usufruire come identificatori mentre l'entità Utente mantiene la propria chiave naturale (email).
3. **Ottimizzazione delle Ridondanze:** All'interno della classe di associazione Partecipazione abbiamo ritenuto più opportuno tenere l'attributo derivato /SaldoCorrente. Dato che, per nostra ipotesi, in un sistema come questo la lettura del saldo rappresentava l'operazione in assoluto più frequente per l'utente; memorizzare il dato in Partecipazione evita al DBMS di dover effettuare calcoli ogni qual volta si richiede di controllarli
4. **Ristrutturazione degli Attributi Multi-valore:** Nel modello concettuale iniziale, avevamo Ritrovato opportuno doverci salvare all'interno della classe Partecipazione degli attributi che conservassero i vari debiti e crediti maturati per singolo utente (DebitiMaturati e CreditiMaturati). Perciò per la loro ristrutturazione. Gli attributi multi valore sono stati eliminate dalle loro classi originarie e sono stati invece promossi a nuove entità autonome, chiamate StoricoDebito e StoricoCredito, fornite di identificatori propri e collegate tramite un'associazione uno-a-molti (1:N) a Partecipazione, che ci permette di tracciare ogni singolo record separatamente, Abbiamo inoltre ritenuto opportune aggiungere una relazione fra le due entità storico debito e storico credito con l'entità quota essendo che sono le singole quote a stabilire la quantità esatta di debito o credito di uno specifico utente, inoltre la relazione fra gli storici e la quota farà in modo che non possano esistere credito o debiti che non abbiano una quota.

4.2 Class Diagram Ristrutturato



4.2 Tabella Formale dei Vincoli (Regole Aziendali)

Di seguito viene presentata la classificazione formale dei vincoli del sistema aggiornata.

Categoria Vincolo	Classe / Associazione Coinvolta	Descrizione Analitica del Vincolo
Vincolo di Dominio	Spesa (TipoSpesa)	L'attributo può assumere esclusivamente i valori appartenenti all'insieme enumerato {PERSONALE, COMUNE}.
Vincolo di Dominio	Invito (StatoInvito)	L'attributo può assumere esclusivamente i valori appartenenti all'insieme enumerato {InAttesa, Accettato, Rifiutato}.
Vincolo di Dominio	Spesa, Quota, Movimento, StoricoDebito, StoricoCredito	I rispettivi attributi che indicano delle somme monetarie, come ImportoTotale e Importo

		devono essere strettamente positivi (> 0).
Vincolo di Dominio	Utente (email)	L'indirizzo e-mail deve terminare obbligatoriamente con il dominio '@studenti.unina.it' oppure '@gmail.com'.
Vincolo interrelazionale	Partecipazione / StoricoDebito / StoricoCredito	L'attributo derivato SaldoCorrente in Partecipazione deve essere sempre pari alla somma degli importi di tutte le istanze collegate di StoricoCredito meno la somma degli importi di tutte le istanze collegate di StoricoDebito del rispettivo utente
Vincolo Intrarelazionale	Utente (Email)	L'indirizzo e-mail costituisce l'identificatore naturale primario e deve essere univoco su tutta l'estensione della classe.
Vincolo Intrarelazionale	ID (Entità varie)	Gli attributi IDGruppo, IDSpesa, IDMovimento, IDDebito e IDCredito devono essere rigorosamente univoci per ogni istanza essendo identificatori delle rispettive entità.
Vincolo Interrelazionale	Invito — Partecipazione	Un record nella classe di associazione Partecipazione per una specifica coppia (Utente, Gruppo) può essere istanziato SE E SOLO SE esiste la corrispondente istanza di Invito con l'attributo StatoInvito = 'Accettato'.

Vincolo Interrelazionale	Quota — Spesa	Un'istanza della classe Quota può essere generata in associazione ad una Spesa solo se l'attributo Spesa.TipoSpesa = 'COMUNE'.
Vincolo Interrelazionale	Gruppo — Partecipazione	L'utente identificato come Creatore di un gruppo deve

		possedere obbligatoriamente un'istanza di Partecipazione attiva per quel medesimo gruppo
Vincolo Interrelazionale	Invito — Gruppo	Essendo l'invito una prerogativa esclusiva del proprietario del gruppo, il mittente di ogni istanza di Invito coincide sempre, in modo implicito e normalizzato, con l'utente associato al gruppo tramite la relazione Crea (ovvero l'attributo EmailCreatore nella tabella Gruppo).
Vincolo Interrelazionale	Movimento — Quota	Un Movimento di rimborso può essere registrato solo se l'utente debitore possiede una Partecipazione attiva con stato accettato (l'invito) all'interno dello stesso gruppo in cui è stata registrata la spesa originaria.
Vincolo Interrelazionale	Saldatura	Un Movimento può saldare una Quota solo se il mittente (chi effettua il rimborso) e il destinatario (il creditore originario che ha anticipato la spesa) appartengono allo stesso identico contesto di Gruppo.

Vincolo Interrelazionale	Spesa — Gruppo	La data di una Spesa non può essere antecedente alla data di creazione del Gruppo in cui la spesa è stata registrata
Vincolo Interrelazionale	Movimento — Spesa	La data di un Movimento (Rimborso) non può essere antecedente alla data della spesa a cui la relativa quota fa riferimento

5. Progettazione Logica (Modello Relazionale)

5.0 Mapping

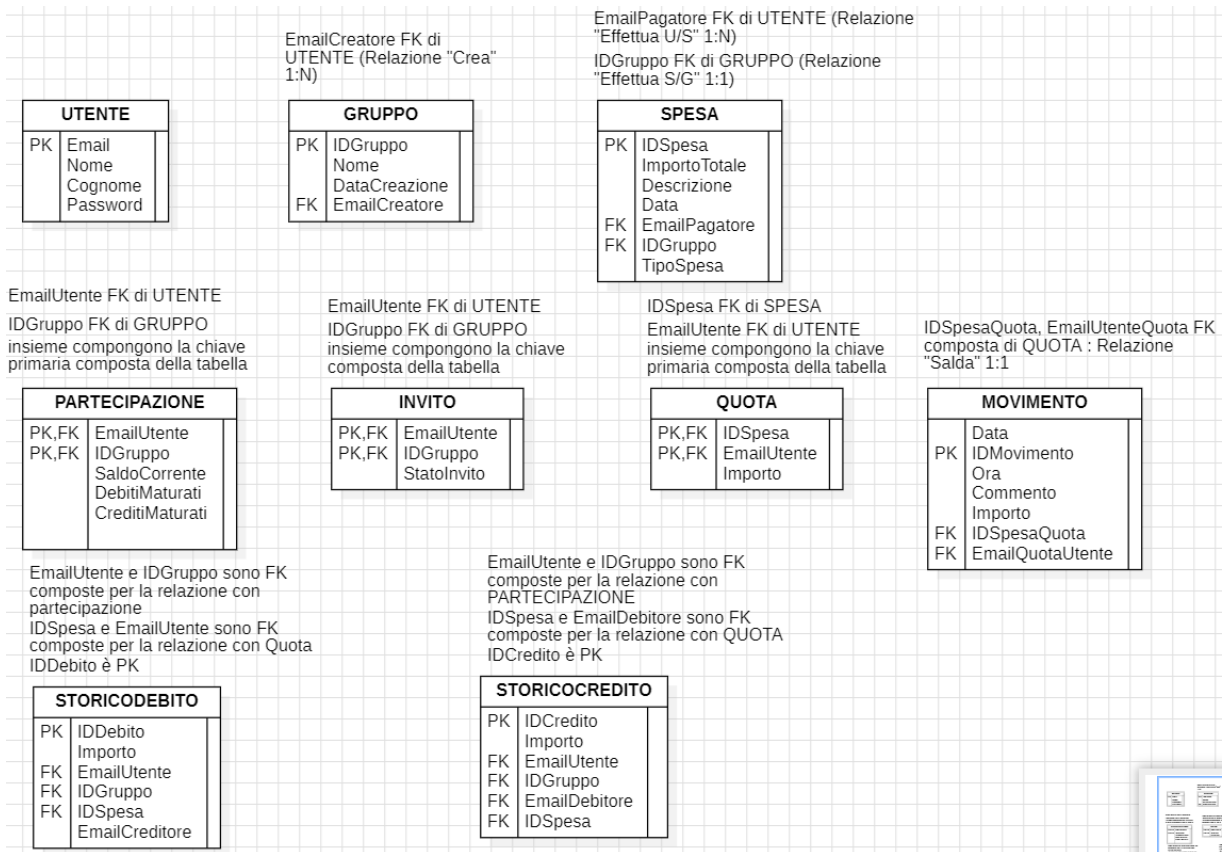
Grazie alla ristrutturazione appena fatta ci è adesso immediato fare il mapping dallo schema concettuale a quello logico, tradurremo ogni entità in una tabella mentre per le associazioni, migrando le chiavi primarie in chiavi esterne o creando le opportune tabelle ponte

5.1 Logica di Traduzione delle Associazioni

- Associazioni Molti-a-Molti (N:M): Le classi di associazione PARTECIPAZIONE, INVITO e QUOTA diventano delle entità autonome. Le loro chiavi primarie sono di tipo composto, generate dall'unione delle chiavi esterne delle entità che connettono.
- Associazioni Uno-a-Molti (1:N): Relazioni come Crea, Effettua Spesa e le relazioni di corrispondenza tra partecipa e gli storici si traducono mediante la tecnica della propagazione della chiave. La chiave primaria del lato con 1 viene inserita come colonna di Foreign Key (FK) nel lato con N
- Associazione Uno-a-Uno (1:1) Quota-Movimento: Poichè dal lato Quota abbiamo una molteplicità 0,1 mentre dal lato Movimento è 1,1, la regola di mapping prevede lo spostamento delle chiavi esterne nell'entità debole. Pertanto, la tabella MOVIMENTO assorbe la chiave della quota a cui fa riferimento.
- Associazione Uno-a-Uno (1:1) StoricoDebito-Quota e StoricoCredito-Quota: Proprio come per l'associazione 1,1 Quota-Movimento, abbiamo optato per spostare la chiave primaria di Quota(1) in StoricoDebito(0,1) e StoricoCredito(0,1), Inoltre in StoricoCredito consideriamo EmailDebitore come l'email dell'utente che è in debito all'utente a cui è stato generato il credito, mentre in StoricoDebito consideriamo EmailCreditore come l'email dell'utente a cui deve essere effettuato il rimborso

5.2 Schema Logico Definitivo (DDL Testuale)

Convenzione di Lettura: I campi contrassegnati con (PK) indicano la Chiave Primaria. I campi contrassegnati con (FK) indicano una Chiave Esterna.



6. Schema Fisico

6.1 Introduzione e Convenzioni

In questo capitolo esporremo le definizioni delle componenti SQL implementate sul DBMS PostgreSQL. Per motivi di sintesi e di leggibilità del documento, verranno riportate soltanto le tabelle principali e un sottoinsieme delle funzioni procedurali (Trigger) più significative a livello di logica di Business. Per l'intero schema fisico, si rimanda al file SQL completo allegato. Le convenzioni di nomenclatura adottate per lo schema fisico sono le seguenti:

- **chk_...:** identifica un vincolo di integrità di tipo CHECK oppure un trigger utilizzato per la validazione di regole di business.
- **auto_...:** identifica i trigger volti ad automatizzare i processi di inserimento, inserendo i dati in modo trasparente all'utente.
- **genera_.../aggiorna_...:** identifica procedure che aggiornano campi derivati o smistano logicamente i dati in base a variazioni di stato.

6.2 Principali Componenti SQL (Tabelle)

Vengono di seguito mostrate le tre tabelle fondamentali del Sistema con relative vincoli di dominio

Tabella Utente

```
CREATE TABLE public.utente (  
    email varchar(100) NOT NULL primary key,  
    nome varchar(100) NOT NULL,  
    cognome varchar(100) NOT NULL,  
    password varchar(100) NOT NULL  
)  
  
alter table utente add constraint chk_email  
check (email like '%@studenti.unina.it' or email like '%@gmail.com')
```

Tabella Spesa

```
create table Spesa(  
    IDSpesa SERIAL primary key,  
    ImportoTotale decimal(10, 2) not null check (ImportoTotale > 0),  
    Descrizione varchar(255),  
    Data date not null default CURRENT_DATE,  
    TipoSpesa varchar(100) not null check (TipoSpesa in ('PERSONALE','COMUNE')),  
    EmailPagatore varchar(255) not null,  
    IDGruppo integer not null,  
    foreign key (EmailPagatore) references utente (email),  
    foreign key (IDGruppo) references gruppo (idgruppo)  
)
```

Tabella Movimento

```
create table movimento(  
    idmovimento serial primary key,  
    data date not null default CURRENT_DATE,  
    ora time not null default CURRENT_TIME,  
    commento varchar(255),  
    importo decimal(10, 2) not null check (importo > 0.00),  
    idspesaquota integer not null,  
    emailquotautente varchar(255) not null,  
    foreign key (idspesaquota, emailquotautente) references quota(idspesa,emailutente)  
)
```

Tabella Gruppo

```
create table gruppo(  
  IDGruppo SERIAL primary key,  
  Nome varchar(100) not null,  
  DataCreazione date DEFAULT current_date,  
  EmailCreatore varchar(255) NOT null,  
  foreign key (EmailCreatore) references utente(Email)  
)
```

Tabella Invito

```
create table invito (  
  emailutente varchar(255),  
  idgruppo integer,  
  statoinvito varchar(100) not null check (statoinvito in ('InAttesa','Accettato','Rifiutato')),  
  primary key (emailutente,idgruppo),  
  foreign key (emailutente) references utente(email),  
  foreign key (idgruppo) references gruppo(idgruppo)  
)
```

Tabella partecipazione

CREATE PARTECIPAZIONE

```
create table partecipazione (  
  EmailUtente varchar(255),  
  IDGruppo integer,  
  SaldoCorrente decimal (10, 2) not null default 0.00,  
  primary key (EmailUtente, IDGruppo),  
  foreign key (EmailUtente) references utente(email),  
  foreign key (IDGruppo) references gruppo(idgruppo)  
)
```

Tabella Quota

CREATE QUOTA

```
create table quota(  
  idspesa integer,  
  emailutente varchar (255),  
  importo decimal(10, 2) not null check (importo > 0.00),  
  primary key (idspesa, emailutente),  
  foreign key (idspesa) references spesa(idspesa),  
  foreign key (emailutente) references utente(email)  
)
```

Tabella Storico Debito

```
CREATE TABLE storicodebito (  
    iddebito SERIAL PRIMARY KEY,  
    importo DECIMAL(10, 2) NOT NULL CHECK(importo > 0.00),  
    emailutente VARCHAR(255) NOT NULL,  
    idgruppo INTEGER NOT NULL,  
    idspesa INTEGER NOT NULL,  
    emailcreditore VARCHAR(255) NOT NULL,  
    FOREIGN KEY (emailutente, idgruppo) REFERENCES partecipazione (emailutente, idgruppo),  
    FOREIGN KEY (idspesa_quota, emailutente) REFERENCES quota (idspesa, emailutente)  
);
```

Tabella Storico Credito

```
CREATE TABLE storicocredito (  
    idcredito SERIAL PRIMARY KEY,  
    importo DECIMAL(10, 2) NOT NULL CHECK(importo > 0.00),  
    emailutente VARCHAR(255) NOT NULL,  
    idgruppo INTEGER NOT NULL,  
    idspesa INTEGER NOT NULL,  
    emaildebitore VARCHAR(255) NOT NULL,  
    FOREIGN KEY (emailutente, idgruppo) REFERENCES partecipazione (emailutente, idgruppo),  
    FOREIGN KEY (idspesa_quota, emaildebitore) REFERENCES quota (idspesa, emailutente)  
);
```

6.2 Principali Componenti SQL (Trigger e Funzioni)

Nel DataBase sono state implementate numerose funzioni e trigger con lo scopo di alleggerire il carico applicativo e per preservarne l'integrità. Vengono qui riportate le due funzioni più rilevanti che fanno largo uso di cursori e di funzioni di aggregazione per aggiornare dinamicamente attributi come saldo e per distribuire automaticamente le quote

Calcolo e Aggiornamento Dinamico del Saldo

la funzione `aggiorna_saldo_function()` calcola la differenza tra il totale maturato in `storicocredito` e `storicodebito` per un utente, andando poi ad aggiornare il campo `saldocorrente` nella tabella `partecipazione`.

```

create or replace function aggiorna_saldo_function()
returns trigger as $$
declare
totale_credito decimal(10,2);
totale_debito decimal(10,2);

begin
select sum(importo)
into totale_credito
from storicocredito
where emailutente = new.emailutente and idgruppo = new.idgruppo;
if totale_credito is null
then totale_credito := 0.00;
end if;
select sum(importo)
into totale_debito
from storicodebito
where emailutente = new.emailutente and idgruppo = new.idgruppo;
if totale_debito is null
then totale_debito := 0.00;
end if;
update partecipazione set saldocorrente = totale_credito - totale_debito
where emailutente = new.emailutente and idgruppo = new.idgruppo;
return new;
end;

$$ language plpgsql;
create trigger aggiorna_saldo_debito
after insert on storicodebito
for each row execute function aggiorna_saldo_function();

create trigger aggiorna_saldo_credito
after insert on storicocredito
for each row execute function aggiorna_saldo_function();

```

Automazione della Generazione Quote

La funzione `auto_genera_quota_function()` fa in modo che quando viene registrata una spesa marcata come 'COMUNE', il trigger recupera tutti i membri del gruppo, calcola l'importo paritario della quota e lo inserisce automaticamente nella tabella quota per ogni singolo membro del gruppo.

Successivamente attraverso un'altra funzione (`genera_storico_quota_function()`) la quota verrà poi automaticamente inserita nella tabella storico credito e debito per i rispettivi creditore e debitore della spesa

```

create or replace function auto_genera_quota function()
returns trigger as $$
declare
num_partecipanti integer;
singola_quota decimal(10,2);
cur_partecipanti cursor for
    select emailutente
    from partecipazione
    where idgruppo = new.idgruppo;
    v_email_partecipante partecipazione.emailutente%type;
begin
if new.tipospesa = 'COMUNE'
THEN
    select count(*)
    into num_partecipanti
    from partecipazione
    where idgruppo = new.idgruppo;
    if num_partecipanti > 0
    then
        singola_quota := round(new.importototale/num_partecipanti,2);
        open cur_partecipanti;
        loop
            fetch cur_partecipanti into v_email_partecipante;
            exit when not found;
            insert into quota (idspesa,emailutente,importo)
            values (new.idspesa,v_email_partecipante,singola_quota);
        end loop;
        close cur_partecipanti;
    end if;
end if;
return new;
end;
$$ language plpgsql;

```